개발환경 및 프로젝트 구성 안내

이 안내는 학습 시작 전 준비용입니다. 설치·다운로드 시간은 14시간 학습 시간에서 제외합니다. WPF는 Windows에서 실행하는 .NET UI 기술입니다. 이 교재는 .NET 10과 Visual Studio 2026을 기준으로 합니다.

1. 필요한 도구 준비

1. Visual Studio 다운로드에서 Visual Studio 2026을 설치합니다. 개인 학습에서는 사용 조건에 맞는 Community 에디션을 선택할 수 있습니다.
2. Visual Studio Installer의 **워크로드**에서 **.NET 데스크톱 개발**을 선택합니다.
3. 개별 구성 요소에서 .NET 10 SDK가 포함되어 있는지 확인하고 설치합니다.
4. Visual Studio를 실행합니다.

이미 설치되어 있다면 Visual Studio Installer에서 해당 Visual Studio의 **수정**을 눌러 워크로드를 확인합니다. SQL Server Management Studio(SSMS) 설치만으로는 WPF 개발환경이 준비되지 않습니다.

이 환경 선택은 Microsoft의 WPF 프로젝트 생성 안내의 Visual Studio 2026·.NET 데스크톱 개발·.NET 10 구성을 기준으로 합니다.

.NET Framework는 기존 Windows 프로그램에서 많이 사용하던 계열입니다. 이 교재는 최신 .NET 계열의 .NET 10을 사용하므로 프로젝트 생성 시 **WPF 애플리케이션(.NET Framework)** 대신 C# **WPF 애플리케이션 / WPF Application**을 선택합니다. 이 차이 외의 역사나 마이그레이션은 학습하지 않습니다.

2. 제공 프로젝트로 시작하기 — 권장

1. 파일 탐색기에서 저장소의 sample/WpfTestApp.sln을 더블클릭합니다.
2. 솔루션 탐색기에 WpfTestApp 프로젝트와 MainWindow.xaml이 보이는지 확인합니다.
3. F5를 누릅니다. 현재 제공 프로젝트는 4주차 완료본이므로 입력창, 실행 버튼, 결과 표가 표시됩니다. 첫 빌드에는 CommunityToolkit.Mvvm 8.4.2 패키지 복원이 필요합니다.
4. 기본 API URL을 그대로 두고 실행하면 게시글이 표에 표시됩니다. 인터넷 연결이 필요합니다.
5. 창을 닫아 실행을 종료합니다.

제공 상태는 **4주차 4일차 완료본(day16)**입니다. 1일차부터 따라가려면 실행을 종료하고 다음 순서로 바꿉니다. 단계 적용 후 1~2주차에는 외부 패키지가 필요 없습니다.

1. Visual Studio의 **보기 → 터미널**을 엽니다. 현재 위치를 저장소 루트(Tutorial_WPF)로 맞춥니다.
2. powershell -ExecutionPolicy Bypass -File .₩sample₩Use-Stage.ps1 -Stage 1을 실행합니다. 교재 파일을 자동 백업하고 해당 단계의 소스와 프로젝트 설정을 함께 적용합니다.
3. Visual Studio가 파일 변경을 물으면 다시 불러옵니다.
4. F5로 실행하여 "첫 번째 WPF 창입니다."가 나타나는지 확인합니다.

날짜별 파일은 시작 코드가 아니라 **그날의 완료본**입니다. 2일차 수업은 1일차 완료 상태에서 시작합니다. stages 폴더 자체를 프로젝트 안에 넣지 않습니다. 스크립트 대신 파일 탐색기로 적용하려면 예제 안내의

전체 파일 복사 절차를 사용하세요.

1주차 완료 상태는 Stage 4, 2주차 완료는 Stage 8, 3주차 완료는 Stage 12, 최종 완료는 Stage 16입니다. 모든 수업은 sample/WpfTestApp 프로젝트에서 이어갑니다. 3주차 3일차부터 패키지 설치가 필요하며 해당 교재에 절차가 있습니다.

3. 직접 새 프로젝트를 만드는 경우

제공 프로젝트로 시작했다면 이 절차는 건너뛰어도 됩니다. 직접 만들더라도 학습에는 프로젝트 하나만 사용합니다.

1. Visual Studio 시작 화면에서 **새 프로젝트 만들기**를 선택합니다.
2. 검색창에 wpf를 입력하고 언어를 **C#**으로 선택합니다.
3. **WPF 애플리케이션 / WPF Application** 템플릿을 고릅니다.
4. 프로젝트 이름을 반드시 WpfTestApp으로 정합니다. 저장 위치는 제공 예제와 다른 빈 폴더로 선택합니다.
5. 프레임워크는 **.NET 10.0**을 선택합니다.
6. 생성 후 F5로 빈 창이 실행되는지 확인하고 닫습니다.
7. 1일차 교재에서 MainWindow 두 파일을 교체하며 시작합니다.

이름을 통일하면 XAML의 x:Class="WpfTestApp.MainWindow"와 C#의 namespace WpfTestApp이 바로 맞아떨어집니다. 프로젝트 이름과 네임스페이스를 바꾸는 방법은 이번 학습에서 생략합니다.

4. 지금 알아둘 프로젝트 파일

파일	쉬운 설명	이번 주에 할 일
WpfTestApp.csproj	대상 .NET과 WPF 사용 여부 등 프로젝트 설정	열어 보기만 함
App.xaml	프로그램 전체의 시작 설정. StartupUri로 첫 창 지정	수정하지 않음
App.xaml.cs	프로그램 전체에 대한 C# 코드	수정하지 않음
MainWindow.xaml	화면의 구성과 배치	수정
MainWindow.xaml.cs	그 화면에 연결된 C# 동작 코드	수정
WpfTestApp.sln	Visual Studio에서 프로젝트를 묶어 여는 파일	더블클릭하여 열기

실행 흐름은 "프로그램 시작 → App의 StartupUri 확인 → MainWindow 생성 → InitializeComponent로 화면 초기화 → 사용자의 입력과 클릭 대기"입니다. 콘솔 앱처럼 우리가 작성한 Main 메서드부터 보이지 않아도 실행 시작 부분은 빌드 과정에서 준비됩니다.

bin, obj 폴더는 빌드가 생성합니다. 교재에서 수정하지 않습니다.

5. 실행에 문제가 있을 때 확인할 최소 항목

증상	확인할 내용
WPF 템플릿이나 .NET 10이 보이지 않음	Visual Studio Installer에서 .NET 데스크톱 개발과 .NET 10 SDK 확인
.NET 10 대상을 지원하지 않는다는 오류	Visual Studio 2026과 .NET 10 SDK로 열었는지 확인
InitializeComponent나 컨트롤 이름을 찾을 수 없음	MainWindow 두 파일의 날짜, x:Class·namespace, x:Name 철자를 확인하고 솔루션 다시 빌드
버튼 이벤트 메서드를 찾을 수 없음	XAML의 Click 값과 C# 메서드 이름이 같은지 확인
실행 버튼이 다른 프로젝트를 가리킴	솔루션 탐색기에서 WpfTestApp 우클릭 → 시작 프로젝트로 설정
XAML 미리보기가 열리지 않음	XAML 텍스트 편집 화면에서 진행하고 빌드·F5 실행 결과 확인

XAML을 변경하기 전에는 실행 중인 창을 닫으세요. 교재는 핫 리로드 설정에 의존하지 않습니다.

터미널 확인은 선택 사항입니다. 저장소 루트에서 다음 명령을 사용할 수 있습니다.

```
dotnet --list-sdks
dotnet build .₩sample₩WpfTestApp.sln
dotnet run --project .₩sample₩WpfTestApp₩WpfTestApp.csproj
```

준비가 끝나면 1주차 1일차로 이동합니다.